

Some of the benefits of object-oriented programming are probably not apparent at this point. Remember that OOP was devised to cope with the complexity of large programs. Smaller programs, such as the examples in this chapter, have less need for the organizational power that OOP provides. The larger the program, the greater the benefit. But even for small programs, once you start thinking in object-oriented terms, the OO design approach becomes natural and surprisingly helpful. One advantage is that in an OO program the compiler can find many more conceptual errors than in a procedural program.

Summary

A class is a specification or blueprint for a number of objects. Objects consist of both data and functions that operate on that data. In a class definition, the members—whether data or functions—can be *private*, meaning they can be accessed only by member functions of that class, or *public*, meaning they can be accessed by any function in the program.

A member function is a function that is a member of a class. Member functions have access to an object's private data, while non-member functions do not.

A constructor is a member function, with the same name as its class, that is executed every time an object of the class is created. A constructor has no return type but can take arguments. It is often used to give initial values to object data members. Constructors can be overloaded, so an object can be initialized in different ways.

A destructor is a member function with the same name as its class but preceded by a tilde (~). It is called when an object is destroyed. A destructor takes no arguments and has no return value.

In the computer's memory there is a separate copy of the data members for each object that is created from a class, but there is only one copy of a class's member functions. You can restrict a data item to a single instance for all objects of a class by making it *static*.

One reason to use OOP is the close correspondence between real-world objects and OOP classes. Deciding what objects and classes to use in a program can be complicated. For small programs, trial and error may be sufficient. For large programs, a more systematic approach is usually needed.

Questions

Answers to these questions can be found in Appendix G.

1. What is the purpose of a class definition?
2. A _____ has the same relation to an _____ that a basic data type has to a variable of that type.

3. In a class definition, data or functions designated `private` are accessible
 - a. to any function in the program.
 - b. only if you know the password.
 - c. to member functions of that class.
 - d. only to public members of the class.
4. Write a class definition that creates a class called `leverage` with one private data member, `crowbar`, of type `int` and one public function whose declaration is `void pry()`.
5. True or false: Data items in a class must be private.
6. Write a statement that defines an object called `lever1` of the `leverage` class described in Question 4.
7. The dot operator (or class member access operator) connects the following two entities (reading from left to right):
 - a. A class member and a class object
 - b. A class object and a class
 - c. A class and a member of that class
 - d. A class object and a member of that class
8. Write a statement that executes the `pry()` function in the `lever1` object, as described in Questions 4 and 6.
9. Member functions defined inside a class definition are _____ by default.
10. Write a member function called `getcrow()` for the `leverage` class described in Question 4. This function should return the value of the `crowbar` data. Assume the function is defined within the class definition.
11. A constructor is executed automatically when an object is _____.
12. A constructor's name is the same as _____.
13. Write a constructor that initializes to 0 the `crowbar` data, a member of the `leverage` class described in Question 4. Assume that the constructor is defined within the class definition.
14. True or false: In a class you can have more than one constructor with the same name.
15. A member function can always access the data
 - a. in the object of which it is a member.
 - b. in the class of which it is a member.
 - c. in any object of the class of which it is a member.
 - d. in the public part of its class.
16. Assume that the member function `getcrow()` described in Question 10 is defined outside the class definition. Write the declaration that goes inside the class definition.

17. Write a revised version of the `getcrow()` member function from Question 10 that is defined outside the class definition.
18. The only technical difference between structures and classes in C++ is that _____.
19. If three objects of a class are defined, how many copies of that class's data items are stored in memory? How many copies of its member functions?
20. Sending a message to an object is the same as _____.
21. Classes are useful because they
 - a. are removed from memory when not in use.
 - b. permit data to be hidden from other classes.
 - c. bring together all aspects of an entity in one place.
 - d. can closely model objects in the real world.
22. True or false: There is a simple but precise methodology for dividing a real-world programming problem into classes.
23. For the object for which it was called, a `const` member function
 - a. can modify both `const` and non-`const` member data.
 - b. can modify only `const` member data.
 - c. can modify only non-`const` member data.
 - d. can modify neither `const` nor non-`const` member data.
24. True or false: If you declare a `const` object, it can only be used with `const` member functions.
25. Write a declaration (not a definition) for a `const void` function called `aFunc()` that takes one `const` argument called `jerry` of type `float`.

Exercises

Answers to the starred exercises can be found in Appendix G.

- *1. Create a class that imitates part of the functionality of the basic data type `int`. Call the class `Int` (note different capitalization). The only data in this class is an `int` variable. Include member functions to initialize an `Int` to 0, to initialize it to an `int` value, to display it (it looks just like an `int`), and to add two `Int` values.

Write a program that exercises this class by creating one uninitialized and two initialized `Int` values, adding the two initialized values and placing the response in the uninitialized value, and then displaying this result.
- *2. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.